



STAYNEX

Ultimate Product, Engineering, Design and Execution Blueprint

International hospitality commerce infrastructure originating in Calabar

Local wedge. International architecture. Real booking engine.

Prepared for plan.md, CTO handoff, and POC v0.1 execution

Version 1.0

Prepared by Bespoke Technologies Think Tank

Brand colors: Blue Royale #27187D and Ghost White #F7F7FF

www.bespoketech.com.ng

@bespoketech_ | 08088071657

DOCUMENT CONTROL

Staynex Blueprint

Staynex is the selected product name for Bespoke Technologies hospitality booking infrastructure. This document defines what we are building, why it matters, how it should behave, what technology should power it, and what the first proof of concept must prove.

This blueprint is not a casual idea note. It is the operating document that should guide plan.md, CTO execution, design systems, engineering decisions, founder pitches, and early partner conversations.

Field	Decision
Product name	Staynex
Parent company	Bespoke Technologies
Origin market	Calabar, Nigeria
Ambition	International hospitality platform
Initial build	Proof of Concept v0.1
Primary standard	Industry-grade from Day 1
Core engine	Property -> Room -> Availability -> Booking -> Payment -> Confirmation

Strategic correction: Staynex is not a Calabar platform. It is an international stay-booking platform originating in Calabar. Calabar is the launch wedge, not the ceiling.

BLUEPRINT MAP

Table of Contents

No.	Section
1	Strategic thesis
2	Product definition
3	Operating principles
4	Service model
5	User groups
6	Core feature blueprint
7	Pages and route map
8	Design system
9	Architecture
10	Data model
11	Booking engine
12	Payments and payouts
13	AI assistant
14	Security and trust
15	Admin operations
16	POC scope
17	30/60/90 roadmap
18	Metrics
19	Risks
20	plan.md starter

1. STRATEGIC THESIS

Why Staynex Should Exist

Hospitality is trust plus availability plus payment plus experience. The customer does not only need a hotel name. They need confidence that the room exists, the price is real, the date is available, the location is reachable, the booking is confirmed, and the property owner has received the right signal.

The opportunity is not simply that many hotels have poor websites. That is only the visible symptom. The deeper problem is that many properties operate without modern booking infrastructure. Staynex should become the reliable connection layer between guests, properties, payments, communications, and verified stay data.

Bespoke Technologies should start from Calabar because the team understands the city, has relationship access, can physically verify early properties, and can use December demand as a natural market test. But every technical and design decision must be prepared for multi-city, multi-country, multi-currency expansion.

The winning thesis: start local enough to learn fast, build global enough to survive scale.

What Must Be True

- Guests must be able to discover stays quickly and trust what they see.
- Property owners must be able to manage availability without technical stress.
- The booking engine must prevent double-booking by design, not by hope.
- Payments, confirmations, notifications, and support must feel coordinated.
- The platform must use clean architecture so new cities, countries, and services can be added without rewriting the product.

2. PRODUCT DEFINITION

What We Are Building

Staynex is a premium, multi-tenant hospitality booking platform for hotels, apartments, short-lets, resorts, serviced apartments, guest houses, villas, and future stay categories. It gives guests a clean way to search, preview, reserve, pay, confirm, communicate, and receive support. It gives property owners a controlled dashboard for property setup, room management, availability, bookings, payouts, and performance.

Staynex is not a directory. A directory lists places. Staynex must confirm stays.

Layer	Meaning
Marketplace	Guests discover and book stays
Booking engine	Availability, holds, payments
Owner OS	Rooms, prices, calendar, bookings
Trust layer	Verification, reviews, support

Layer	Meaning
AI layer	Guidance, support, recommendations
Admin layer	Approvals, disputes, fees, controls

Scope Boundary

The first version should prove the full commercial loop. It should not attempt to build every travel feature. Flights, events, tours, restaurants, loyalty, and white-label APIs are future expansion layers. The first POC must ship the end-to-end booking loop cleanly.

3. OPERATING PRINCIPLES

Principles That Control the Product

Principle	Decision
Real before fancy	Booking works first
Simple first	No feature noise
Secure by default	Low friction, strong control
Expansion-ready	No Calabar-only schema
Cards everywhere	Visual, fast scanning
AI with boundaries	Helpful, not reckless
Owner-friendly	Simple hotel operations
Admin-visible	Every critical action traceable
Daily vertical slices	Ship usable progress
Trust beats hype	Verified stays win

The lowest standard is industry standard. Staynex should feel like a serious technology product from the first demo, even if the feature count is small.

4. SERVICE MODEL

Services Staynex Will Provide

Staynex begins as a booking platform, but the strategic architecture must allow it to become a hospitality commerce system. Each service should either help guests book confidently, help owners operate better, or help Bespoke Technologies compound capability.

Service	Phase	Purpose
Stay booking	POC	Core revenue loop
Property onboarding	POC	Supply creation
Owner dashboard	POC	Availability control
Admin console	POC	Trust and operations
AI stay assistant	Phase 2	Guided discovery
Reviews	Phase 2	Trust signals
Loyalty	Phase 3	Repeat usage
Corporate booking	Phase 3	Higher value accounts
White-label engine	Phase 4	Infrastructure revenue
Hotel website upsell	Anytime	Bespoke service revenue

5. USER GROUPS

The Three-Sided Platform

Staynex must be designed as a three-sided platform. The guest experience brings demand. The owner experience brings supply. The admin experience protects the platform.

Guest Jobs

- Search stays by destination and date.
- Preview rooms visually before committing.
- Check real-time availability.
- Reserve now or plan future dates.
- Pay securely and receive confirmation.
- Get location, policy, and support guidance.

Owner Jobs

- Register and verify property.
- Create rooms, prices, media, and amenities.
- Manage calendar availability.

- Receive bookings and notifications.
- Track earnings and payout status.
- Communicate with platform support.

Admin Jobs

- Approve properties and owners.
- Monitor bookings and payments.
- Handle disputes and cancellations.
- Control featured listings and city settings.
- Review audit logs and AI actions.
- Protect guest and owner trust.

6. CORE FEATURE BLUEPRINT

Feature Scope by Product Stage

The core feature strategy is simple: first prove the complete booking loop, then add trust, intelligence, automation, and expansion capabilities.

Stage	Guest	Owner	Admin
POC	Search, view, book, pay	Rooms, images, calendar	Approve, monitor
MVP	Accounts, history, saves	Earnings, messages	Fees, disputes
V1	Reviews, AI help	Analytics, promos	Reports, policy
Scale	Multi-currency	Channel tools	Country controls

POC Non-Negotiables

- Three sample properties with realistic rooms and images.
- Real database-backed property and room records.
- Availability checks by city, dates, and guest count.
- Swipeable card gallery for property and room previews.
- Booking creation with a transaction-safe hold.
- Paystack test payment and payment webhook handling.
- Email confirmation through Resend.
- Owner dashboard booking visibility.
- Admin dashboard booking and property visibility.
- Mobile-first public interface.

7. PAGES AND ROUTE MAP

Public, Owner, and Admin Surfaces

Every page must share the same component logic, design tokens, spacing system, motion rules, and error handling. The platform must not feel like three separate products stitched together.

Surface	Routes
Public	/, /search, /stay/[slug]
Booking	/checkout, /payment, /confirmed
Guest	/guest, /guest/bookings
Owner	/owner, /owner/properties
Owner Rooms	/owner/rooms, /owner/calendar

Surface	Routes
Owner Money	/owner/earnings, /owner/payouts
Admin	/admin, /admin/properties
Admin Ops	/admin/bookings, /admin/payments
Admin Trust	/admin/disputes, /admin/audit
Legal	/terms, /privacy, /cancellation

8. DESIGN SYSTEM

Visual Identity and Interface System

Staynex should look premium without becoming decorative. The interface must make booking feel calm, guided, visual, and trustworthy. The uploaded color direction is adopted as a controlled foundation: Blue Royale and Ghost White.

Token	Value	Usage
Blue Royale	#27187D	Primary brand
Ghost White	#F7F7FF	Base surface
Ink	#111827	Primary text
Muted	#5B6472	Secondary text
Line	#E4E7EC	Borders
Success	#0E9F6E	Confirmed
Amber	#B7791F	Pending
Error	#C81E1E	Failed

Card-Based UI

The card is the main product surface. A Staynex card should carry image, location, trust, price, availability, and action without visual clutter. The image area should support horizontal swipe, subtle gradient border, and full-screen preview. This gives users a modern, Instagram-familiar interaction while keeping the platform professional.

Property Card Elements

- Swipeable image carousel.
- Subtle Blue Royale gradient border.
- Property name and location.
- Price from label.
- Verified badge.
- Availability badge.
- Rating and review count.
- Key amenities.
- Save and quick-view actions.

Motion Rules

- Motion must guide attention, not perform.
- Cards may lift softly on hover.

- Image swipes should be smooth and quick.
- Loading states should use skeletons.
- Success states should feel final and calm.
- No noisy bounce effects.

9. TECHNOLOGY ARCHITECTURE

Industry-Grade Stack From Day 1

The architecture should start as a disciplined modular platform, not a fragile prototype. The correct early choice is a modular monolith with clean domain boundaries, strong database transactions, strict validation, and deployable vertical slices. Microservices can come later when operational complexity justifies them.

Layer	Technology	Role
Web	Next.js + TS	Public and dashboards
API	NestJS + TS	Domain backend
Database	CockroachDB	Distributed SQL
Storage	Cloudflare R2	Images and docs
Deploy Web	Vercel	Frontend hosting
Deploy API	Railway	Backend hosting
AI	Gemini + AI SDK	Assistant layer
Maps	Google Maps	Location
Auth	Google OAuth	Identity
Email	Resend	Transactional email
Push	Firebase	Notifications
Support	Respond.io	WhatsApp later

Current official docs support the stack choices: CockroachDB transactions provide ACID semantics across distributed data, Cloudflare R2 exposes S3-compatible object storage, the Vercel AI SDK is a TypeScript toolkit for AI-powered applications, and Gemini function calling connects models to external tools and APIs. These references are listed at the end of this blueprint.

Monorepo Recommendation

```

staynex/
  apps/web          # Next.js public, guest, owner, admin
  apps/api          # NestJS API
  packages/ui       # shared components
  packages/types    # shared TS contracts
  packages/config   # eslint, tsconfig, env schemas
  packages/db       # migrations and schema helpers
  docs/            # blueprint, plan.md, specs

```

10. DATA MODEL BLUEPRINT

Entities That Must Exist

The schema must never assume a single city. Every property must belong to a city, region, country, currency, and owner. Room types and room units must be separated so the platform can support both simple hotel inventory and advanced room-level control later.

Domain	Entities
Identity	User, Role, Session
Profiles	GuestProfile, OwnerProfile
Location	Country, Region, City
Property	Property, Verification
Rooms	RoomType, RoomUnit
Media	PropertyMedia, RoomMedia
Pricing	RatePlan, FeeRule
Availability	Calendar, Hold
Booking	Booking, BookingGuest
Money	Payment, Payout, Refund
Trust	Review, Dispute
Comms	Message, Notification
AI	AIConversation, AIActionLog
Ops	AuditLog, SupportTicket

Critical Data Rule

Booking creation must be transactional. The system must check availability, create a temporary hold, start payment, confirm booking after payment success, and release expired holds automatically. No confirmed booking should exist without verified availability and verified payment state.

11. BOOKING ENGINE

The Heart of Staynex

The booking engine is the product. The homepage is presentation. The AI is assistance. The admin console is control. But the engine is what converts trust into revenue.

Step	Status
Search	query

Step	Status
Availability	available/unavailable
Hold	pending_payment
Payment	initiated
Webhook	verified
Booking	confirmed
Notify	sent
Check-in	validated
Payout	eligible

Anti-Double Booking Rules

- Use database transactions for hold creation.
- Expire unpaid holds after a fixed window.
- Never show stale availability without rechecking.
- Prevent overlapping bookings on the same room unit or inventory count.
- Record every admin override in AuditLog.
- Notify owner and admin when inventory conflicts appear.

12. PAYMENTS AND PAYOUTS

Money Flow and Commercial Model

Staynex should start with Paystack for Nigeria-first payments, while keeping the payments domain abstract enough to add Stripe, Flutterwave, bank transfer, wallet, corporate invoice, and local processors later.

Public language matters. Unless the legal structure is fully defined, avoid calling it escrow in user-facing copy. Use platform-managed payout, held payout, or payout after booking validation. The product behavior can still protect guests and owners with controlled payout timing.

Item	POC Decision
Payment provider	Paystack test mode
Currency	NGN first
Commission	Configurable
Payout	Manual first
Refund	Admin-controlled
Receipt	Email receipt
Webhook	Verified only

Revenue Streams

- Commission per confirmed booking.
- Featured property placement.
- Hotel website upsell by Bespoke Technologies.
- Premium owner dashboard later.
- Corporate booking package later.
- White-label booking engine later.

13. AI ASSISTANT

Platform-Aware, Not Generic

The Staynex AI assistant must know the product, policies, properties, rooms, amenities, guest context, owner context, and support process. It must be useful without pretending to be the platform authority for financial, legal, or irreversible actions.

AI Can	AI Must Not
Recommend stays	Invent availability
Explain policies	Promise refunds

AI Can	AI Must Not
Guide booking	Confirm payment
Compare rooms	Change records alone
Draft support replies	Reveal private data
Help owners onboard	Override rules

AI Action Model

AI should suggest, explain, retrieve, and prepare actions. The platform should verify, authorize, and execute. This keeps the assistant helpful without making it dangerous.

14. SECURITY, TRUST, AND COMPLIANCE

Secure Without Feeling Rigid

Staynex must use strong security invisibly. The guest should not feel punished for booking, and owners should not feel blocked from operating. The system should make safe behavior the default.

Area	Minimum Standard
Transport	HTTPS
Auth	OAuth + sessions
Access	RBAC
Data	Tenant isolation
Inputs	Validation
Payments	Webhook verify
Media	Signed uploads
Ops	Audit logs
Secrets	Env hygiene
Recovery	Backups

Trust Features

- Verified property badges.
- Photo approval before publishing.
- Clear cancellation policy.
- Transparent price breakdown.
- Booking status history.
- Review moderation.
- Support ticket trail.
- Owner response time tracking.

15. ADMIN OPERATIONS

The Control Tower

The admin console must be built early because trust, support, property verification, payments, disputes, and growth cannot be managed from the database or chat messages. Admin visibility is part of the product, not a back-office luxury.

Admin Module	Purpose
Dashboard	System overview

Admin Module	Purpose
Properties	Approve listings
Bookings	Monitor flow
Payments	Verify money
Payouts	Owner settlement
Disputes	Resolve issues
Reviews	Moderate trust
Cities	Expansion control
Fees	Commercial rules
AI Logs	Assistant safety
Audit Logs	Accountability

16. PROOF OF CONCEPT V0.1

What the First Build Must Prove

The POC must not try to impress with feature count. It must prove that Staynex can complete a real booking loop with clean data, visible dashboards, secure payment flow, and a beautiful mobile-first user experience.

POC Success Statement

A guest can search a city, view a property card, swipe room images, confirm availability, book a room, pay in test mode, receive confirmation, and the owner/admin can see the booking instantly.

POC Slice	Acceptance
Search	City + dates
Property card	Swipe images
Room detail	Images + amenities
Availability	Real check
Booking	DB record
Payment	Paystack test
Email	Resend sent
Owner	Booking visible
Admin	Full trace
Mobile	Clean UX

17. EXECUTION ROADMAP

30/60/90 Day Plan

Window	Focus	Outcome
Day 1-7	Foundation	Repo, schema, tokens
Day 8-14	Supply	Owner + rooms
Day 15-21	Booking	Search + payment
Day 22-30	Demo	POC ready
Day 31-60	MVP	Reviews, messages
Day 61-90	Pilot	Hotels + guests

Daily Shipping Rhythm

- Every day must improve one vertical slice.

- No isolated feature work without UI/API/data connection.
- Keep a working demo at all times.
- Use seeded data for demos from the first week.
- Record technical debt explicitly in plan.md.

18. METRICS AND RISKS

How We Know Staynex Is Working

Metric Type	Metrics
Product	Search, view, book
Money	GBV, commission
Supply	Verified rooms
Trust	Disputes, reviews
Ops	Response time
Growth	Repeat guests

Important Risks

Risk	Control
Stale availability	Calendar + holds
Double booking	Transactions
Bad photos	Approval rules
Payment disputes	Clear policy
Owner adoption	Simple dashboard
Scope creep	POC gate
AI mistakes	Tool boundaries
Legal ambiguity	Careful wording

19. PLAN.MD STARTER

Implementation Plan Skeleton

This blueprint should now be translated into plan.md. The plan.md should be task-oriented, technical, and updated daily. It should not repeat the full strategy. It should convert the strategy into work units, owners, dependencies, acceptance criteria, and current status.

```
# Staynex plan.md

## Mission
Build an international hospitality booking platform originating in Calabar.

## Current Phase
POC v0.1 - Property -> Room -> Availability -> Booking -> Payment -> Confirmation.

## Stack
Next.js, NestJS, TypeScript, CockroachDB, Cloudflare R2, Vercel, Railway,
Paystack, Resend, Firebase, Google Maps, Google OAuth, Gemini, Vercel AI SDK.

## Workstreams
- Design system
```

- Web app
- API
- Database
- Media storage
- Booking engine
- Payments
- Notifications
- Admin
- Owner dashboard
- AI assistant
- QA and security

Week 1

- [] Create monorepo
- [] Configure env validation
- [] Define design tokens
- [] Create database schema
- [] Setup auth foundation
- [] Setup R2 upload flow
- [] Seed sample cities and properties

Week 2

- [] Owner onboarding
- [] Property creation
- [] Room creation
- [] Image carousel card
- [] Admin approval
- [] Availability calendar

Week 3

- [] Search results
- [] Availability check
- [] Booking hold
- [] Paystack test payment
- [] Payment webhook
- [] Confirmation page

Week 4

- [] Resend email
- [] Owner booking dashboard
- [] Admin booking dashboard
- [] AI prototype
- [] Mobile polish
- [] Demo readiness report

20. TECHNICAL REFERENCE ANCHORS

Current Documentation Anchors

These references are not included to copy product strategy. They are included to ground key technology assumptions in official or primary documentation.

Reference	URL
CockroachDB transactions	https://www.cockroachlabs.com/docs/stable/transactions
CockroachDB architecture	https://www.cockroachlabs.com/docs/stable/architecture/overview
Cloudflare R2 overview	https://developers.cloudflare.com/r2/
Cloudflare R2 S3 API	https://developers.cloudflare.com/r2/api/s3/api/
Vercel AI SDK	https://ai-sdk.dev/docs/introduction
Gemini function calling	https://ai.google.dev/gemini-api/docs/function-calling

Founder-Level Summary

Staynex should be built as a serious hospitality infrastructure company, not a local website. The POC should prove the core booking loop. The design must be premium and repeatable. The architecture must be expansion-ready. The AI must be useful but bounded. The admin layer must protect trust. The product must ship daily until the platform becomes real.